Co-Robot Sewing Simulation - 기술 문서

1. 프로젝트 개요

Co-robot 1대와 재봉기 2대 또는 3대의 통합 작업 흐름을 시뮬레이션하는 웹 애플리케이션. 순수 HTML/CSS/JavaScript로 구현되어 별도 빌드 도구나 프레임워크 없이 동작합니다. 시작 시 **모드 선택 화면**에서 재봉기 수량(2대/3대)을 선택한 후 시뮬레이션을 진행합니다.

1.1 시스템 구성

3대 모드:

```
                [재봉기 2]
                    |
                    |
[재봉기 1] ---- [Co-Robot] ---- [재봉기 3]
                    |
                    |
               [자재 테이블]
```

- **재봉기 1**: Co-Robot 좌측
- **재봉기 2**: Co-Robot 상단
- **재봉기 3**: Co-Robot 우측

2대 모드:

```
[재봉기 1] ---- [Co-Robot] ---- [재봉기 2]
                    |
                    |
               [자재 테이블]
```

- **재봉기 1**: Co-Robot 좌측
- **재봉기 2**: Co-Robot 우측

공통:

- **Co-Robot**: 중앙에 위치, 자재 운반 및 재봉기 서비스 담당
- **자재 테이블**: Co-Robot 하단, 새 작업물 공급 및 완료품 수거 장소

1.2 작업 흐름

초기 적재 사이클 (빈 재봉기):

```
자재테이블 ---(초기적재시간)---> 재봉기X: 새 작업물 load ---(복귀시간)──→ 자재테이블
                                 재봉기X: 재봉 시작
                                 (생산량 증가 없음)
                                 대기시간 = 시뮬레이션 시작(0초)부터 load 완료 시점까지
                                 (다른 재봉기 초기 적재 + 복귀 시간이 포함됨)
```

정상 사이클 (재봉 완료된 재봉기):

```
자재테이블 ---(UL&Load시간)---> 재봉기X: 완료품 unload → 생산량+1
                                          + 새 작업물 load ---(복귀시간)──→ 자재테이블:
                                 재봉기X: 재봉 시작
                                 대기시간 누적 종료
```

2. 파일 구조

```
COROBOT/
├── index.html              # 메인 HTML (모드 선택, 레이아웃, 입력 폼, 시뮬레이션 영역, 메뉴
├── image/
│   ├── icon_robot.png      # 로봇 아이콘 이미지 (시뮬레이션 영역 표시용)
│   └── pallet.png          # 자재 테이블 아이콘 이미지
├── DOCUMENT.md             # 이 문서
├── css/
│   └── style.css           # 전체 스타일 + CSS 애니메이션 + 모드 선택 스타일 + 반응형
├── js/
│   ├── config.js            # 설정값 관리 (입력값 읽기/검증, 재봉기 수량 관리, 로봇 속도 데
│   ├── engine.js            # 이벤트 기반 시뮬레이션 엔진 (핵심 로직, Pallet 큐 시스템)
│   ├── quickCalc.js         # 빠른계산 모드 (렌더링 없이 즉시 실행)
│   ├── renderer.js          # DOM 기반 시각화 렌더러 (동적 좌표 지원, Pallet 정보)
│   ├── realtimeCalc.js      # 실시간계산 모드 (RAF 루프, 타임라인 seek, cargo 시각화)
│   ├── charts.js            # Chart.js 기반 결과 차트 (생산량/대기시간 막대, 가동률 도넛)
│   ├── scenario.js          # 시나리오 비교 기능 (저장/삭제/비교 테이블/차트)
│   ├── pdf.js               # PDF 리포트 내보내기 (jsPDF + autoTable, 동적 모드 지원)
│   └── main.js              # 앱 초기화, 모드 선택, 이벤트 바인딩, UI 컨트롤러
└── eng/                     # 영문 버전
    ├── index.html
    └── js/
        ├── config.js
        ├── engine.js
        ├── quickCalc.js
        ├── renderer.js
        ├── realtimeCalc.js
        ├── charts.js
        ├── scenario.js
        ├── pdf.js
        └── main.js
```

2.1 모듈 의존성 다이어그램

```
index.html
  ├── css/style.css
  │
  ├── js/config.js ──────────────────────────────> 입력값 검증/제공 + 재봉기 수량 관리
  │       |
  │   js/engine.js ──────────────────────────────> 시뮬레이션 엔진 (핵심)
  │       |                          |
  │   js/quickCalc.js    js/realtimeCalc.js
  │       | (결과 표시)              |
  │   js/renderer.js <──────────────┘ (DOM 업데이트)
  │       |
  │   js/charts.js ───────────────────────────────> Chart.js 결과 시각화
  │       |
  │   js/scenario.js ─────────────────────────────> 시나리오 비교 기능
  │
  │   js/pdf.js ──────────────────────────────────> PDF 리포트 내보내기
  │
  └── js/main.js ─────────────────────────────────> 컨트롤러 (모드 선택 + 모드 전환)
```

2.2 스크립트 로드 순서 (index.html)

```
<!-- 외부 라이브러리 (CDN) -->
<script src="https://cdn.jsdelivr.net/npm/jspdf@2.5.1/dist/jspdf.umd.min.js"></scrip
<script src="https://cdn.jsdelivr.net/npm/jspdf-autotable@3.8.2/dist/jspdf.plugin.au
<script src="https://cdn.jsdelivr.net/npm/chart.js@4.4.1/dist/chart.umd.min.js"></sc

<!-- 내부 모듈 (순서 중요) -->
<script src="js/config.js"></script>
<script src="js/engine.js"></script>
<script src="js/quickCalc.js"></script>
<script src="js/renderer.js"></script>
<script src="js/realtimeCalc.js"></script>
<script src="js/charts.js"></script>
<script src="js/scenario.js"></script>
<script src="js/pdf.js"></script>
<script src="js/main.js"></script>
```

3. 사용자 입력값

3.0 기본 설정

시뮬레이션 전체에 적용되는 공통 설정입니다.

항목	ID	설명	기본값
작업인원	`worker-count`	PPH 계산에 사용되는 인원 수 (1~4명)	2명
Pallet 준비 시간	`pallet-prep-time`	작업자가 새 원단을 준비하는 시간 (초)	58초
Pallet 수량	`pallet-count-options`	시스템에서 순환하는 총 Pallet 개수 (3~7개, 버튼 선택)	2대 모드: 5개, 3대 모드: 6개

Pallet 수량 UI:

- 3/4/5/6/7개 버튼 중 하나를 클릭하여 선택
- `Config.readBasicSettings()` 에서 `.pallet-count-btn.active` 버튼의 값을 읽음
- 모드 변경 시 `setPalletCountDefault()` 함수가 모드별 기본값 자동 설정

3.1 재봉기별 작업 시간

각 재봉기별로 아래 4가지를 초 단위로 입력합니다.

#	항목	ID 패턴	설명
1	초기 Load 시간	`m{N}-init`	자재테이블에서 pick → 빈 재봉기에 load (unload 없음)
2	Unload & Load 시간	`m{N}-ul`	자재테이블에서 pick → 재봉기 완료품 unload + 새 작업물 load
3	복귀(회수) 시간	`m{N}-return`	재봉기에서 완료품 pick → 자재테이블로 복귀
4	재봉 시간	`m{N}-sewing`	재봉기가 작업물을 재봉하는 데 걸리는 시간

- **2대 모드**: 8개 입력 (4항목 x 2대, 재봉기 1·2)
- **3대 모드**: 12개 입력 (4항목 x 3대, 재봉기 1·2·3)

HTML 기본값 (재봉기별 개별 설정, 85% 속도 기준):

재봉기	초기 Load	UL&Load	복귀(회수)	재봉
M1	11초	15.5초	9초	63초
M2	12초 (11.8→반올림)	16.5초 (16.3→반올림)	10초 (9.8→반올림)	63초
M3	11초	15.5초	9초	63초

- JS 코드 내 파싱 실패 시 대체값: 초기적재 15초, UL&Load 20초, 복귀 10초, 재봉 60초

3.2 시뮬레이션 시간 설정

시뮬레이션을 실행할 총 시간을 선택합니다.

항목	ID	설명	기본값
시간	`sim-hours`	1~24시간 선택	1시간
분	`sim-minutes`	0~59분 선택 (24시간 선택 시 00분만 가능)	00분

- 총 시뮬레이션 시간은 `시간 × 3600 + 분 × 60` 초로 계산
- 빠른계산과 실시간계산 모두 이 설정을 사용

3.3 로봇 속도 선택

작업 시간(초기 Load, UL&Load, 복귀)은 로봇 속도에 따라 달라집니다. **재봉 시간은 로봇 속도와 무관**하므로 속도 선택과 관계없이 항상 수동 입력합니다.

선택 옵션	동작
직접입력	사용자가 모든 값을 수동 입력
75%	하드코딩된 75% 속도 값 자동 입력
80%	75%~85% 구간 선형 보간 (ratio 0.5)
85%	하드코딩된 85% 속도 값 자동 입력 **(기본값)**
90%	85%~100% 구간 선형 보간 (ratio 0.33)
95%	85%~100% 구간 선형 보간 (ratio 0.67)
100%	하드코딩된 100% 속도 값 자동 입력

하드코딩 값 (75% 속도):

재봉기	초기 Load	UL&Load	복귀
M1	12.5초	17.5초	10.5초
M2	13.5초	18.5초	11.5초
M3	12.5초	17.5초	10.5초

하드코딩 값 (85% 속도) - 기본값:

재봉기	초기 Load	UL&Load	복귀
M1	11초	15.5초	9초
M2	11.8초	16.3초	9.8초
M3	11초	15.5초	9초

하드코딩 값 (100% 속도):

재봉기	초기 Load	UL&Load	복귀
M1	8.5초	13초	7.5초
M2	9초	13.5초	8초
M3	8.5초	13초	7.5초

보간 공식 (구간별):

- 75~85% 구간: `값 = 값_75% + (값_85% - 값_75%) × (속도% - 75) / 10`
- 85~100% 구간: `값 = 값_85% + (값_100% - 값_85%) × (속도% - 85) / 15`
- 결과는 소수점 1자리까지 반올림 (`Math.round(value * 10) / 10`)
- 앱 시작 시 기본 속도 85%가 자동 적용되며 input 필드가 readonly 상태로 설정됨
- 속도 선택 시: 초기 Load, UL&Load, 복귀 input이 **readonly** 상태로 자동 입력됨
- 직접입력 선택 시: 모든 input의 readonly 해제

4. 핵심 모듈 상세

4.1 config.js - 설정 관리

```
Config = {
    MACHINE_IDS: [1, 2, 3],      // 동적 변경됨 (2대 모드: [1,2], 3대 모드: [1,2,3])
    setMachineCount(count),       // 재봉기 수량 설정 → MACHINE_IDS 업데이트
    getMachineCount(),            // 현재 재봉기 수량 반환
    getConfig(),                  // 입력값 읽기 + 유효성 검증 → { valid, errors, machi
    getTheoreticalMax(),          // 이론 최대 생산량 = floor(totalSeconds / sewingTim
    readBasicSettings(),          // 기본 설정 읽기 (workerCount, palletPrepTime, pall
    SPEED_DATA,                   // 75%/85%/100% 하드코딩 속도 데이터
    getSpeedValues(speedPercent)  // 속도(%)에 따른 작업 시간값 반환 (구간별 선형 보간)
}
```

getSpeedValues(speedPercent) 동작:

- `speedPercent`가 75, 85, 100이면 `SPEED_DATA`에서 직접 반환
- 75~85% 구간: 75%와 85% 사이를 선형 보간하여 계산
- 85~100% 구간: 85%와 100% 사이를 선형 보간하여 계산
- 반환값: `{ 1: {initLoadTime, unloadLoadTime, returnTime}, 2: {...}, 3: {...} }`
- 소수점 1자리까지 반올림하여 반환
- 재봉시간(sewingTime)은 포함하지 않음 (로봇 속도와 무관)

readBasicSettings() 반환값:

```
{
    workerCount: parseInt('worker-count' value) || 2,
    palletPrepTime: parseFloat('pallet-prep-time' value) || 0,
    palletCount: parseInt('.pallet-count-btn.active' data-pallet-count) || 5,
}
```

MACHINE_IDS 동적 변경:

- `setMachineCount(2)` 호출 시 `MACHINE_IDS` → `[1, 2]`
- `setMachineCount(3)` 호출 시 `MACHINE_IDS` → `[1, 2, 3]`
- 배열을 in-place로 수정 (`length=0` + `push`)하여 모든 모듈의 참조가 자동 반영됨

4.2 engine.js - 시뮬레이션 엔진 (핵심)

이벤트 기반 이산 사건 시뮬레이션(Discrete Event Simulation) 엔진입니다. 빠른계산과 실시간계산이 동일한 엔진을 공유하여 결과의 일관성을 보장합니다. `Config.MACHINE_IDS`를 순회하므로 2대/3대 모드에서 별도 수정 없이 동작합니다.

상태 정의

로봇 상태:

상태	설명
`AT_TABLE_IDLE`	자재테이블에서 대기 중 (다음 작업 결정)
`TRAVELING_TO_MACHINE`	작업물 들고 재봉기로 이동 중
`TRAVELING_TO_TABLE`	완료품 들고 테이블로 복귀 중

로봇 추가 플래그:

플래그	설명
`carryingCompleted`	완료품을 가지고 있는지 (boolean)
`isInitialLoad`	초기 적재 중인지 (boolean)
`isUnloadOnly`	Unload-only 모드인지 (boolean)

재봉기 상태:

상태	색상	설명
`EMPTY`	회색	초기 상태, 작업물 없음
`SEWING`	초록	재봉 중 (진행률 표시)
`DONE_WAITING`	주황	재봉 완료, 로봇 대기 중 (깜빡임)

이벤트 타입

이벤트	발생 시점	처리
`ROBOT_READY_AT_TABLE`	로봇이 테이블 도착	스케줄링 알고리즘 실행
`ROBOT_ARRIVED_AT_MACHINE`	로봇이 재봉기 도착	unload(생산량+1)/load 완료, 재봉 시작
`ROBOT_RETURNED_TO_TABLE`	로봇이 테이블 복귀	완료품 도착 기록, 다음 스케줄링
`MACHINE_DONE_SEWING`	재봉 완료	DONE_WAITING 전환, 로봇 트리거

스케줄링 알고리즘

로봇이 테이블에 있을 때 다음 목적지를 결정하는 우선순위:

```
1순위: DONE_WAITING 상태인 재봉기 → 가장 오래 기다린 재봉기 우선
2순위: EMPTY 상태인 재봉기 → 번호순 (1→2 또는 1→2→3) 초기 적재
3순위: 모두 SEWING 중 → 가장 먼저 끝날 재봉기 완료까지 대기
```

Pallet 큐 시스템

Pallet 수량에 따른 자재 준비 로직을 관리합니다.

상태 변수:

변수	설명
`readyPalletsQueue`	준비 완료된 pallet의 준비 시간 배열 (정렬됨)
`extraPalletsRemaining`	아직 준비 시작 안 한 추가 pallet 개수
`totalPickups`	테이블에서 pallet pick up 횟수

초기화 (reset):

- 초기 준비 pallet = 재봉기 수량 (모두 t=0에 준비 완료)
- 추가 pallet = `palletCount - machineCount` (아직 준비 시작 안함)

Pallet 흐름:

```
[초기 상태: 재봉기 3대, Pallet 5개]
readyPalletsQueue = [0, 0, 0]  (3개 준비 완료)
extraPalletsRemaining = 2      (추가 2개 대기)

[M1 load 시작]
readyPalletsQueue = [0, 0]     (1개 사용)
extraPalletsRemaining = 1      (추가 pallet 1개 준비 시작)
readyPalletsQueue = [0, 0, t+prepTime]

[재봉 완료 후 unload → 테이블 도착]
→ 빈 pallet에 자재 준비 시작
→ readyPalletsQueue에 (현재시간 + prepTime) 추가
```

Unload-only 모드:

Pallet이 없지만 DONE_WAITING 재봉기가 있을 때 사용됩니다.

상황	동작
Pallet 있음 + DONE_WAITING	정상 UL&Load 수행
Pallet 없음 + DONE_WAITING	Unload-only 모드로 출발
Pallet 없음 + EMPTY만	Pallet 준비 대기

- 빈 손으로 재봉기에 이동 (returnTime 사용)
- unload만 수행, load 안함
- 재봉기는 EMPTY 상태로 전환
- 완료품을 들고 테이블로 복귀
- 테이블 도착 시 빈 pallet에 자재 준비 시작

주요 메서드

```
class SimulationEngine {
    constructor(machineConfigs, palletPrepTime, palletCount)  // 설정값 + Pallet 준비
    reset()                        // 상태 초기화
    start()                        // 시뮬레이션 시작 (첫 이벤트 등록)

    // 시뮬레이션 진행
    runUntil(endTime)              // 지정 시간까지 모든 이벤트 처리 (빠른계산)
    advanceTo(upToTime)            // 지정 시간까지 이벤트 처리 (실시간)

    // 상태 조회
    getState()                     // 현재 상태 스냅샷 (palletInfo 포함)
    getResults()                   // 최종 통계 (생산량, 대기시간, 가동률, 가동시간, totalPi

    // 스케줄링
    scheduleNext()                 // 다음 목적지 결정
    findEarliestSewingDone()       // 가장 먼저 끝날 재봉기 시점

    // 콜백
    onEvent                        // 이벤트 발생 시 호출 (실시간 로그)
    onStateChange                  // 상태 변경 시 호출 (렌더러 업데이트)
}
```

getState() 반환 구조:

```
{
    time, robot, machines,
    robotBusyTime,
    palletInfo: {
        readyCount,        // 현재 준비된 pallet 수
        prepRemaining,     // 다음 pallet 준비 완료까지 남은 시간 (초)
        isPreparing,       // 현재 준비 중인 pallet이 있는지
    }
}
```

통계 계산 방식

통계	계산 방식
생산량	로봇이 재봉기에서 완료품을 unload할 때 +1 (초기 적재 제외)
대기시간	초기 적재: 시뮬레이션 시작(0초)부터 load 완료까지. 정상 사이클: (로봇 도착하여 load하는 시점) - (재봉 완료 시점). 사이클마다 누적
로봇 가동률	(로봇 가동시간 합계) / (총 시뮬레이션 시간) x 100%
로봇 총 가동시간	로봇이 이동 중인 시간의 합계 (초), `getResults().robotBusyTime`
이론 최대	floor(총 시뮬레이션시간 / 재봉시간)
재봉기 총 가동시간	모든 재봉기의 실제 재봉 시간 합계 (완료 사이클 + 진행 중), `getResults().totalMachineSewingTime`
재봉기 총 가동률	(재봉기 총 가동시간) / (시뮬레이션시간 × 재봉기 수) × 100%, `getResults().machineUtilization`
PPH	(총 생산수량 / 시뮬레이션시간(h) / 작업인원), Person Per Hour
Pallet Pickup 횟수	로봇이 테이블에서 pallet을 집어간 횟수, `getResults().totalPickups`

4.3 quickCalc.js - 빠른계산

```
QuickCalc = {
    run(machineConfigs, palletPrepTime, palletCount),  // 설정된 시간 동안 엔진 실행 → 
    displayResults(results, workerCount),              // DOM에 결과 렌더링 (PPH 계산 
    setDuration(seconds),                              // 시뮬레이션 시간 설정 (기본 360
    getDuration(),                                     // 현재 설정된 시뮬레이션 시간 반
}
```

- 엔진을 렌더링 없이 즉시 실행
- `setDuration()` 으로 설정된 시간(기본 3600초)을 시뮬레이션
- 결과: 각 재봉기별 생산수량/대기시간, 이론 최대값, 로봇 가동률, 로봇 총 가동시간, PPH, 재봉기 총 가동률/가동시간
- 이벤트 로그 테이블 자동 생성
- `displayResults()` 내부에서 `Charts.updateCharts(results)` 호출하여 차트 자동 업데이트

- `Config.MACHINE_IDS`를 순회하므로 2대/3대 모드에서 별도 수정 없이 동작

4.4 renderer.js - 시각화 렌더러

```
Renderer = {
    POSITIONS: { ... },              // 시뮬레이션 영역 내 좌표 (% 기반, 모드에 따라 동적
    init(),                          // DOM 요소 캐싱
    updateState(state),              // 전체 상태 업데이트 (Pallet 정보 포함)
    updateElapsedTime(time),         // 경과 시간 표시
    resetView(),                     // 초기 상태로 리셋 (Pallet 정보 포함)
    setSkipRobotPosition(val),       // 실시간 모드 로봇 위치 제어 플래그
    setMachineCount(count),          // 재봉기 수량에 따라 POSITIONS 업데이트
}
```

updateState(state) 처리 순서:

1. `updateElapsedTime(state.time)` - 경과 시간 표시
2. `updateRobotStatus(state.robot)` - 로봇 상태 텍스트
3. `updateRobotPosition(state.robot)` - 로봇 위치 (skipRobotPosition=false일 때만)
4. `updateMachines(state.machines)` - 재봉기 상태/진행률/색상
5. `updateRealtimeStats(state.machines)` - 실시간 통계 바
6. `updatePalletInfo(state.palletInfo)` - Pallet 준비 상태

Pallet 정보 표시:

- `pallet-ready-count`: 현재 준비된 pallet 수
- `pallet-timer`: 준비 중인 pallet의 남은 시간 (있을 때만 표시)

시뮬레이션 영역 좌표 (3대 모드):

```
M2: (50%, 22%)      ← 상단 (M2 bottom edge 근처)
M1: (12%, 50%)      ← 좌측
M3: (88%, 50%)      ← 우측
Table: (50%, 82%)   ← 하단 (로봇 대기 위치)
```

시뮬레이션 영역 좌표 (2대 모드):

```
M1: (12%, 50%)      ← 좌측
M2: (88%, 50%)      ← 우측 (3대 모드의 M3 위치로 이동)
Table: (50%, 82%)   ← 하단 (로봇 대기 위치)
```

setMachineCount(count) 동작:

- 2대 모드: `POSITIONS[2]` → `{x:88, y:50}` (우측), `POSITIONS[3]` 삭제
- 3대 모드: `POSITIONS[2]` → `{x:50, y:22}` (상단), `POSITIONS[3]` → `{x:88, y:50}` (우측)
- POSITIONS 객체를 in-place로 수정하여 realtimeCalc 등 참조하는 모듈이 자동 반영됨
- 로봇 아이콘: `image/icon_robot.png` 이미지 사용 (CSS `filter: brightness(0) invert(1)` 로 흰색 변환)
- 자재 테이블 아이콘: `image/pallet.png` 이미지 사용
- 로봇은 대기 시 자재테이블 위치에 배치 (z-index: 5로 테이블 노드 위에 표시)

4.5 realtimeCalc.js - 실시간 시뮬레이션

```
RealtimeCalc = {
    start(machineConfigs, palletPrepTime, palletCount),  // 시뮬레이션 시작 + RAF 루프
    pause(),                  // 일시정지 (RAF 중단, 상태 보존)
    resume(),                 // 다시시작 (RAF 재개, 시간 점프 방지)
    stop(),                   // 멈춤 → 최종 결과 표시, results 반환
    setSpeed(newSpeed),       // 속도 변경 (0.1x ~ 100x)
    getSpeed(),               // 현재 속도
    isActive(),               // 실행 중 여부
    setDuration(seconds),     // 시간 제한 설정 (0 = 무제한)
    setOnTimeLimit(callback),// 시간 제한 도달 시 콜백 설정
    setOnTimeUpdate(callback),// 매 프레임 시간 업데이트 콜백 설정
    seekTo(targetTime),       // 특정 시점으로 이동 (되감기/건너뛰기)
    getSimTime(),             // 현재 시뮬레이션 시간
    getMaxDuration(),         // 설정된 최대 시간
}
```

애니메이션 루프 (tick):

1. 실제 경과시간 x 속도배수 = 시뮬레이션 시간 증분
2. 시간 제한 체크 (`maxDuration > 0 && simTime ≥ maxDuration` → 자동 정지)
3. `engine.advanceTo(simTime)` → 이벤트 처리
4. `Renderer.updateState(state)` → 재봉기 상태, 통계, Pallet 정보 업데이트
5. 로봇 총 가동시간/대기시간 실시간 업데이트
6. 타임라인 업데이트 콜백 호출
7. 로봇 애니메이션 보간 (easeInOutCubic 이징) + 이동 경과시간 표시

로봇 애니메이션:

- 엔진 이벤트 콜백(`handleEngineEvent`)으로 출발/도착 트리거

- 자재테이블 → 재봉기 → 자재테이블 경로로 이동
- fromPos → toPos를 이징 함수로 부드럽게 보간
- 초기적재/정상사이클 시간 자동 구분
- 이동 중 경과시간 표시 (예: `→ M1 (3.2초)`), 새 이동 시 0부터 리셋
- `Renderer.POSITIONS`를 참조하므로 2대/3대 모드에서 별도 수정 없이 동작

Cargo (운반물) 시각화:

- `cargoState`: `'none'` | `'new'` | `'done'`
- 새 pallet 운반 시: 파란색 "P" 배지 (`carrying-new` 클래스)
- 완성품 운반 시: 금색 "✓" 배지 (`carrying-done` 클래스)
- Unload-only 출발 시: 배지 없음 (빈 손)
- `updateCargoDisplay()`에서 `#robot-cargo` / `#cargo-icon` DOM 업데이트

타임라인 Seek:

- `seekTo(targetTime)`: 특정 시점으로 이동
- 뒤로 되감기: 엔진을 새로 생성하고 `targetTime`까지 빠르게 실행 (Re-simulation 방식)
- 앞으로 건너뛰기: `engine.advanceTo(targetTime)` 호출
- seek 중 이벤트 콜백 비활성화 (애니메이션 트리거 방지)
- seek 후 로봇 위치/cargo 상태/렌더러 상태 스냅

4.6 charts.js - 결과 차트

```
Charts = {
    updateCharts(results),    // 4개 차트 모두 업데이트
    destroyAllCharts(),       // 모든 차트 파괴 (모드 변경 시)
}
```

4가지 차트:

차트	유형	Canvas ID	설명
재봉기별 생산량	막대 차트 (Bar)	`chart-production`	각 재봉기의 완성 수량 비교
재봉기별 대기시간	막대 차트 (Bar)	`chart-waittime`	각 재봉기의 로봇 대기 시간 비교
로봇 시간 활용	도넛 차트 (Doughnut)	`chart-robot-util`	가동시간 vs 대기시간, 중앙에 가동률(%)
재봉기 시간 활용	도넛 차트 (Doughnut)	`chart-machine-util`	봉제시간 vs 유휴시간, 중앙에 가동률(%)

재봉기별 색상:

- M1: 파랑 (`rgba(59, 130, 246, 0.7)`)
- M2: 초록 (`rgba(34, 197, 94, 0.7)`)
- M3: 빨강 (`rgba(239, 68, 68, 0.7)`)

도넛 차트 중앙 텍스트:

- `centerText` 플러그인으로 차트 중앙에 가동률(%) 표시
- 로봇: 보라색 (`#7c3aed`), 재봉기: 초록색 (`#22c55e`)

4.7 scenario.js - 시나리오 비교

```
ScenarioManager = {
    MAX_SCENARIOS: 5,                       // 최대 저장 수
    saveScenario(name, settings, results), // 시나리오 저장
    deleteScenario(id),                     // 개별 삭제
    clearAllScenarios(),                    // 모두 삭제
    getScenarios(),                         // 저장된 시나리오 목록
    updateComparisonUI(),                   // 비교 UI 업데이트
    openSaveDialog(settings, results),      // 저장 다이얼로그 열기
    destroyCharts(),                        // 비교 차트 파괴
}
```

시나리오 데이터 구조:

```
{
    id: Date.now().toString(),
    name: "시나리오 이름",
    timestamp: Date.now(),
    mode: Config.getMachineCount(),    // 2 또는 3
    settings: {
        workerCount, palletPrepTime, palletCount,
        simDuration, machines: { ... }
    },
    results: {
        totalProduced, pph, robotUtilization,
        machineUtilization, robotBusyTime,
        totalMachineSewingTime, simTime,
        machines: { [id]: { produced, totalWaitTime } }
    }
}
```

비교 테이블 지표:

- 총 생산량 (높을수록 최적)
- PPH (높을수록 최적)
- 로봇 가동률 (높을수록 최적)
- 재봉기 가동률 (높을수록 최적)
- 총 대기시간 (낮을수록 최적)
- 설정값 비교: 작업인원, Pallet 준비시간, Pallet 수량

비교 차트 (3가지):

차트	유형	Canvas ID
총 생산량 비교	막대 차트	`chart-scenario-production`
PPH 비교	막대 차트	`chart-scenario-pph`
효율성 비교	레이더 차트	`chart-scenario-util`

시나리오별 색상 (최대 5개):

- 파랑, 초록, 주황, 빨강, 보라

4.8 pdf.js - PDF 리포트 내보내기

```
PdfExport = {
    exportPdf(results, machineConfigs)   // PDF 생성 및 다운로드
}
```

PDF 리포트 구성 (5개 섹션):

1. **기본 설정**: 작업인원, Pallet 준비시간, Pallet 수량
2. **입력 설정값**: 재봉기별 작업 시간 테이블
3. **시뮬레이션 결과**: 재봉기별 생산량/대기시간/효율
4. **종합 요약**: 총 생산수량, 로봇 가동률, 로봇 총 가동시간, 시뮬레이션 시간, PPH, 재봉기 총 가동률/가동시간, 재봉기별 효율
5. **이벤트 로그**: 전체 이벤트 로그 (건수 표시)

동적 모드 지원:

- `Config.getMachineCount()` 와 `Config.MACHINE_IDS` 를 사용하여 동적으로 테이블 생성
- `getMachineLabel(id)` : 모드에 따라 위치 레이블 반환 (좌측/우측/상단)
- PDF 제목에 모드 정보 포함: `( 재봉기 2대 모드 )` 또는 `( 재봉기 3대 모드 )`
- 파일명에 모드 포함: `CoRobot_2M_Report_ ... pdf` 또는 `CoRobot_3M_Report_ ... pdf`
- 종합 요약에 재봉기별 효율을 동적으로 추가
- 각 페이지에 푸터: 페이지 번호 + `© SUNTECH 2026`

한글 폰트:

- NanumGothic TTF를 CDN에서 동적 로드 후 캐싱
- `loadKoreanFont()` → `registerFont()` → 모든 autoTable에 `font: 'NanumGothic'` 적용

4.9 main.js - 메인 컨트롤러

```
// 앱 모드
mode = 'idle' | 'quick' | 'realtime'

// 추가 상태
lastResults = null              // 마지막 시뮬레이션 결과 (PDF/시나리오용)
lastMachineConfigs = null       // 마지막 시뮬레이션 설정값 (PDF/시나리오용)
savedM2Values = null            // 2대 모드 전환 시 m2 원본 값 보관용
currentRobotSpeed = 'manual'    // 현재 로봇 속도 선택
```

시뮬레이션 시간 선택

```
getSelectedDuration()           // 선택한 시간을 초 단위로 반환 (hours*3600 + minutes*60)
updateDurationDisplay()         // 총 시뮬레이션 시간 표시 업데이트
populateMinutes(max)            // 분 select 옵션 초기화 (0~max)
```

- 시간 select 변경 시 24시간이면 분 옵션을 0만 표시
- 빠른계산: `QuickCalc.setDuration(getSelectedDuration())` 호출
- 실시간계산: `RealtimeCalc.setDuration(getSelectedDuration())` + `setOnTimeLimit()` 콜백 설정

로봇 속도 선택

```
applySpeedValues(speedValues)   // 속도 값을 input 필드에 적용 (재봉시간 제외)
setInputsReadonly(readonly)     // 초기Load/UL/복귀 input의 readonly 상태 설정
```

속도 제어 (실시간계산)

```
sliderToSpeed(val)              // 슬라이더(0~100) → 속도(0.1~100) 로그 스케일 변환
speedToSlider(speed)            // 속도 → 슬라이더 역변환
formatSpeed(speed)              // 속도 포맷 (0.10x / 1.0x / 10x)
applySpeed(speed)               // 속도 적용 (슬라이더 + 표시 + RealtimeCalc 동기화)
```

- 프리셋 버튼: 0.1x, 0.5x, 1x, 5x, 10x, 50x, 100x
- 로그 스케일 슬라이더: `10^((val/50) - 1)` 공식으로 0.1x~100x 매핑
- 프리셋 클릭 시 슬라이더도 동기화, 슬라이더 드래그 시 프리셋 활성 해제

타임라인 컨트롤

```
initTimeline(duration)          // 타임라인 초기화 (최대값 설정)
updateTimeline(currentTime)     // 매 프레임 타임라인 위치 업데이트
formatTimelineTime(seconds)     // 시간 포맷 (0:00 또는 0:00:00)
```

- `timeline-slider`: range input으로 드래그하여 특정 시점 이동
- `-10s` / `+10s` 버튼: 10초 단위 점프
- 드래그 중(`timelineDragging=true`)이면 tick 루프의 자동 업데이트 차단
- 드래그 완료 시 `RealtimeCalc.seekTo(time)` 호출

모드 선택

```
selectMode(count)       // 모드 선택 처리 - Config/Renderer 업데이트, UI 전환
goToModeSelection()     // 모드 선택 화면으로 복귀 (실행 중 시뮬레이션 중지)
setupSimArena(count)    // 시뮬레이션 아레나 노드 배치 업데이트
openManual()            // 메뉴얼 모달 열기
closeManual()           // 메뉴얼 모달 닫기 (ESC 키/오버레이 클릭도 지원)
```

selectMode(count) 동작:

1. `Config.setMachineCount(count)` → MACHINE_IDS 업데이트
2. `Renderer.setMachineCount(count)` → POSITIONS 업데이트
3. `setPalletCountDefault(count)` → Pallet 수량 기본값 설정
4. UI 전환: 모드 선택 화면 숨김 → 앱 화면 표시
5. 헤더 서브타이틀 업데이트 (재봉기 수량 + 모드 변경 버튼 + 메뉴얼 버튼)
6. 입력 섹션: 2대 모드 시 재봉기 3 입력 숨김, 그리드 2열로 변경
   - 2대 모드: M2 원본 값을 `savedM2Values`에 보관 후, M3 값을 M2 input에 복사
   - 3대 모드: `savedM2Values`에서 M2 원래 값 복원
7. 결과/통계 섹션: 2대 모드 시 재봉기 3 관련 요소 숨김
8. 시뮬레이션 아레나: `setupSimArena(count)` 호출

setupSimArena(count) 동작:

- 2대 모드: M2 노드를 우측(M3 위치)으로 이동, M3 노드 숨김, 상단 SVG 경로 숨김
- 3대 모드: M2 노드를 상단으로 복원, M3 노드 표시, 상단 SVG 경로 표시

시나리오 관리

```
// 저장
btnSaveScenario → ScenarioManager.openSaveDialog(settings, lastResults)
                → updateScenarioCount()

// 삭제
btnClearScenarios → confirm → ScenarioManager.clearAllScenarios()
                  → updateScenarioCount()

// 개수 표시
updateScenarioCount()  // #scenario-count에 (현재/최대) 표시
```

버튼 동작

버튼	동작
2대 모드 / 3대 모드	모드 선택 → selectMode() 호출
(모드 변경)	goToModeSelection() → 모드 선택 화면 복귀
메뉴얼	사용 메뉴얼 모달 표시 (ESC 또는 오버레이 클릭으로 닫기)
로봇 속도 (직접입력/75%~100%)	속도 선택 → 작업 시간 자동 입력 (직접입력: 수동)
빠른계산	Config 검증 → 선택한 시간 설정 → QuickCalc 실행 → 결과+로그+차트 표시
실시간계산	Config 검증 → 시간 제한/타임라인 설정 → RealtimeCalc 시작 → 시뮬레이션 뷰 표시
일시정지	RealtimeCalc.pause() → 애니메이션 일시 중단 (실시간 모드 중에만 표시)
다시시작	RealtimeCalc.resume() → 애니메이션 재개 (일시정지 시에만 표시)
멈춤	RealtimeCalc 중지 → 최종 결과+로그+차트 표시
속도(0.1x~100x)	프리셋 버튼 또는 로그 스케일 슬라이더
-10s / +10s	타임라인에서 10초 단위 이동
시나리오 저장	현재 결과를 시나리오로 저장 (이름 입력)
모두 삭제	저장된 시나리오 모두 삭제 (확인 후)
PDF 저장	시뮬레이션 결과를 PDF 리포트로 내보내기
로그 토글	이벤트 로그 접기/펼치기

버튼 가시성:

- 일시정지/다시시작: 초기 숨김 → 실시간계산 시작 시 일시정지 표시 → 클릭 시 서로 토글 → 멈춤 시 숨김

5. 두 가지 계산 모드

5.1 빠른계산

- 설정된 시간(기본 1시간, 최대 24시간)을 즉시 시뮬레이션
- 렌더링 없이 엔진만 실행 → 수 밀리초 내 완료
- **결과 출력:**
  - 각 재봉기별 총 생산수량
  - 각 재봉기별 총 대기시간 (초)
  - 총 생산수량, 로봇 가동률, 로봇 총 가동시간, 시뮬레이션 시간
  - PPH (Person Per Hour), 재봉기 총 가동률, 재봉기 총 가동시간
  - 결과 차트 4종 (막대 + 도넛)
  - 상세 이벤트 로그 테이블
  - PDF 리포트 저장 가능
  - 시나리오 저장 가능

5.2 실시간계산

- `requestAnimationFrame` 기반 실시간 시각화
- **시간 제한:** 설정된 시뮬레이션 시간에 도달하면 자동 정지
- **속도 조절:** 0.1x ~ 100x (로그 스케일 슬라이더 + 7개 프리셋)
- **일시정지/다시시작:** 시뮬레이션을 일시 중단하고 재개 가능
- **타임라인:** 슬라이더 드래그 또는 ±10초 버튼으로 특정 시점 이동/되감기
- **시각화 요소:**
  - 재봉기 상태별 색상 변화 (회색→초록→주황)
  - 재봉기 진행률 바
  - 로봇 이동 애니메이션 (자재테이블 ↔ 재봉기, easeInOutCubic 이징 적용)
  - 로봇 이동 경과시간 실시간 표시 (새 이동 시 0부터 리셋)
  - Pallet 운반 상태 시각화 (파란색 P 배지 / 금색 ✓ 배지)
  - Pallet 준비 상태 표시 (준비 수량 + 준비 중 남은 시간)
  - 실시간 통계 바 (재봉기별 생산수/대기시간 + 로봇 총 가동시간/대기시간)
- **시간 제한 도달 시:** 자동 정지 → `onTimeLimit` 콜백 실행
- **멈춤 시:** 해당 시점까지의 결과, 차트, 이벤트 로그 표시

6. 검증 방법

6.1 이벤트 로그 검증

빠른계산 실행 시 모든 이벤트의 시간/타입/재봉기/설명이 테이블로 출력됩니다. 수동으로 시간을 추적하여 로직을 검증할 수 있습니다.

로그 형식:

시간(초)	이벤트	재봉기	설명
0.0	INIT	-	시뮬레이션 시작
0.0	ROBOT_DEPART	M1	로봇 → 재봉기 1 (초기 적재, 15초)
15.0	INIT_LOAD	M1	재봉기 1: 초기 적재 완료
...	...	...	...

6.2 교차 검증

- 빠른계산과 실시간계산이 **동일한 엔진(SimulationEngine)**을 사용
- 같은 입력값으로 실행하면 동일 시간대에서 동일한 결과가 나와야 함
- 실시간 시뮬레이션을 1시간까지 실행 후 멈추면 빠른계산과 결과 비교 가능

6.3 이론적 상한 체크

```
각 재봉기 최대 생산량 = floor(3600 / 재봉시간)
실제 생산량 <= 최대 생산량   (항상 성립)
```

- 결과 화면에 이론적 최대값이 함께 표시됨

6.4 로봇 가동률 밸런스 체크

```
로봇 가동시간 = Σ(각 서비스 사이클의 Input1 + Input2)
로봇 유휴시간 = 총 시간 - 로봇 가동시간
로봇 가동률 = 로봇 가동시간 / 총 시간 × 100%
```

6.5 수동 검증 예시 - 3대 모드 (기본값)

모든 재봉기 동일 설정: 초기적재 15초, UL&Load 20초, 복귀 10초, 재봉 60초

```
초기 적재 흐름:
t=0     로봇 출발 → M1 (초기적재 15초)
t=15    M1 적재 완료, 재봉 시작 (60초, 완료 t=75), M1 초기 대기 15초
t=25    로봇 테이블 복귀, 출발 → M2 (초기적재 15초)
t=40    M2 적재 완료, 재봉 시작 (60초, 완료 t=100), M2 초기 대기 40초
t=50    로봇 테이블 복귀, 출발 → M3 (초기적재 15초)
t=65    M3 적재 완료, 재봉 시작 (60초, 완료 t=125), M3 초기 대기 65초
t=75    로봇 테이블 복귀, M1 재봉 완료 (대기 0초)

정상 사이클:
t=75    로봇 출발 → M1 (UL&Load 20초)
t=95    M1 unload → 생산 1개, load, 재봉 시작 (완료 t=155), M1 대기 20초
t=105   로봇 테이블 복귀 (완료품 도착)
t=105   로봇 출발 → M2 (UL&Load 20초), M2 대기 5초
...
정상 상태: 로봇 사이클 = 30초/재봉기 x 3대 = 90초
재봉시간 60초 < 로봇 사이클 90초 → 각 재봉기 매 사이클 30초 대기

1시간 결과: 총 생산 117개 (39+39+39), 로봇 가동률 100%
```

6.6 수동 검증 예시 - 2대 모드 (기본값)

모든 재봉기 동일 설정: 초기적재 15초, UL&Load 20초, 복귀 10초, 재봉 60초

```
초기 적재 흐름:
t=0     로봇 출발 → M1 (초기적재 15초)
t=15    M1 적재 완료, 재봉 시작 (60초, 완료 t=75), M1 초기 대기 15초
t=25    로봇 테이블 복귀, 출발 → M2 (초기적재 15초)
t=40    M2 적재 완료, 재봉 시작 (60초, 완료 t=100), M2 초기 대기 40초
t=50    로봇 테이블 복귀, 대기 (M1·M2 모두 SEWING 중)

정상 사이클:
t=75    M1 재봉 완료, 로봇 출발 → M1 (UL&Load 20초)
t=95    M1 unload → 생산 1개, load, 재봉 시작 (완료 t=155), M1 대기 20초
t=105   로봇 테이블 복귀 (완료품 도착), M2 대기 5초
t=105   로봇 출발 → M2 (UL&Load 20초)
 ...
정상 상태: 로봇 사이클 = 30초/재봉기 x 2대 = 60초
재봉시간 60초 = 로봇 사이클 60초 → 로봇 유휴시간 발생

1시간 결과: 총 생산 88개 (44+44), 로봇 가동률 약 74.9%
```

7. 사용 방법

7.1 실행

1. `index.html` 을 웹 서버를 통해 열기 (또는 직접 브라우저에서 열기)
2. **모드 선택 화면**에서 `재봉기 2대` 또는 `재봉기 3대` 선택
3. **기본 설정** 확인 (작업인원, Pallet 준비시간, Pallet 수량)
4. **로봇 속도** 확인/변경 (기본값 85%, 직접입력 / 75%~100% 선택 가능)
5. **시뮬레이션 시간** 설정 (1~24시간)
6. **빠른계산** 또는 **실시간계산** 클릭

7.2 모드 변경

- 헤더의 **[모드 변경]** 버튼 클릭으로 모드 선택 화면으로 복귀 가능
- 헤더의 **[메뉴얼]** 버튼 클릭으로 사용 설명서 모달 표시
- 실시간 시뮬레이션 실행 중이면 자동으로 중지 후 복귀
- 이전 결과는 초기화됨

7.3 로봇 속도 선택

1. **85%** (기본): 앱 시작 시 자동 적용됨 (입력 필드가 readonly 상태)
2. **속도 버튼 (75%~100%)** 클릭: 초기 Load, UL&Load, 복귀 시간이 자동 입력됨
   - 입력 필드가 readonly 상태로 변경 (파란색 배경)
   - 재봉 시간은 속도와 무관하므로 항상 수동 입력 가능
3. 속도 변경 시 다른 속도 버튼을 클릭하면 즉시 값이 갱신됨
4. **직접입력** 클릭 시 readonly 해제, 이전 자동 입력값을 기반으로 수동 수정 가능

7.4 빠른계산 사용

1. 시간값 입력 후 **빠른계산** 클릭
2. 설정된 시뮬레이션 시간 동안의 결과 즉시 표시
3. 결과 차트에서 시각적으로 확인
4. 이벤트 로그에서 상세 흐름 확인
5. 접기/펼치기 버튼으로 로그 토글

7.5 실시간계산 사용

1. 시간값 입력 후 **실시간계산** 클릭
2. 시뮬레이션 뷰에서 로봇 이동, 재봉기 상태, Pallet 운반 상태 확인
3. 속도 프리셋 버튼(0.1x~100x) 또는 로그 스케일 슬라이더로 속도 조절
4. 타임라인 슬라이더를 드래그하거나 ±10초 버튼으로 특정 시점 이동
5. **일시정지** 클릭으로 시뮬레이션 일시 중단, **다시시작**으로 재개
6. **멈춤** 클릭으로 시뮬레이션 종료 및 결과/차트 확인

7.6 시나리오 비교

1. 시뮬레이션 실행 후 **[시나리오 저장]** 클릭
2. 시나리오 이름 입력 (기본: "시나리오 N")
3. 다른 설정으로 시뮬레이션 실행 후 다시 저장 (최대 5개)
4. 2개 이상 저장 시 자동으로 비교 섹션 표시:
   - 성능 비교표 (최고값 녹색 하이라이트)
   - 총 생산량/PPH 막대 차트
   - 효율성 레이더 차트
5. 개별 삭제(×) 또는 **[모두 삭제]**로 관리

7.7 PDF 리포트

1. 빠른계산 또는 실시간계산 완료 후 **PDF 저장** 클릭
2. 한글 폰트(NanumGothic)를 CDN에서 자동 로드
3. PDF 리포트 구성:
   - 1. 기본 설정 (작업인원, Pallet 준비시간, Pallet 수량)
   - 2. 입력 설정값 테이블 (선택한 모드의 재봉기만 포함)
   - 3. 시뮬레이션 결과 테이블 (재봉기별 생산량/대기시간/효율)
   - 4. 종합 요약 (총 생산수량, 로봇 가동률, 로봇 총 가동시간, 시뮬레이션 시간, PPH, 재봉기 총 가동률/가동시간, 재봉기별 효율)
   - 5. 전체 이벤트 로그
4. 파일명 형식: `CoRobot_{N}M_Report_YYYYMMDD_HHMM.pdf` (N: 재봉기 수량)

8. 기술 스택

항목	기술
마크업	HTML5
스타일	CSS3 (Grid, Flexbox, Custom Properties, Animations, 반응형)
로직	Vanilla JavaScript (ES6+)
시각화	SVG (경로) + DOM (노드) + CSS Transitions
차트	Chart.js 4.4.1 (막대, 도넛, 레이더)
애니메이션	requestAnimationFrame + CSS @keyframes
PDF	jsPDF 2.5 + jspdf-autotable 3.8 + NanumGothic 한글 폰트
프레임워크	없음 (순수 구현)
빌드 도구	없음

9. 주요 설계 결정

결정	이유
이벤트 기반 시뮬레이션	틱 기반보다 정확하고 효율적
단일 엔진 공유	빠른계산과 실시간의 결과 일관성 보장
삽입 정렬 이벤트 큐	이벤트 수가 적어 충분히 효율적
CSS transition 대신 JS 보간	속도 배수 변경에 유연하게 대응
DOM 기반 시각화	Canvas보다 접근성과 상호작용 용이
모듈 패턴 (IIFE)	전역 스코프 오염 방지, 순수 JS 호환
로봇 대기 위치를 자재테이블로	실제 작업 동선과 일치하는 시각화
로봇 아이콘 이미지 사용	이모지 대신 커스텀 이미지로 일관된 표현
일시정지/다시시작 토글 표시	버튼 수를 줄이고 현재 상태를 명확히 전달
MACHINE_IDS in-place 수정	배열 참조를 공유하는 모든 모듈이 자동으로 새 설정 반영
POSITIONS 객체 in-place 수정	동일 참조를 유지하여 realtimeCalc 등에서 자동 반영
모드 선택 화면 분리	시작 시 명확한 모드 선택 UX 제공, 모드 변경 시 안전한 상태 초기화
2대 모드에서 M2를 우측 배치	좌-우 대칭 배치로 직관적인 레이아웃 제공
엔진/quickCalc/realtimeCalc 무수정	Config.MACHINE_IDS 순회 설계 덕분에 모드 변경이 config/renderer/main에만 영향
75%/85%/100% 하드코딩 + 구간별 보간	세 기준점으로 더 정확한 보간, 85%를 기본값으로 설정하여 즉시 사용 가능
재봉시간은 속도 선택에서 제외	재봉 시간은 재봉기 고유 속성이며 로봇 속도와 무관
readonly input으로 자동 입력 표시	사용자가 자동 입력 값과 수동 입력 구간을 시각적으로 구분 가능
시뮬레이션 시간 사용자 선택	1~24시간 범위로 다양한 시나리오 분석 가능, 실시간계산 시간 제한으로 자동 정지
메뉴얼 모달 내장	별도 문서 없이 앱 내에서 사용법 바로 확인 가능
PPH 계산 포함	작업인원 고려한 시간당 생산성 지표로 공정 효율 비교 용이

결정	이유
2대 모드 M2 값 보관/복원	모드 전환 시 M3 값을 M2에 복사하고, 3대 모드 복귀 시 원래 M2 값 복원
Pallet 큐 시스템	단일 nextPalletReadyTime 대신 배열로 관리하여 여러 pallet 동시 추적 가능
초기 pallet = 재봉기 수량	시작 시 각 재봉기에 즉시 load 가능한 상태 보장
Unload-only 모드	Pallet이 없어도 DONE_WAITING 재봉기 서비스 가능, 데드락 방지
추가 pallet 준비 시점	기존 pallet load 시작 시 준비 시작하여 자재 준비 선행 가능
Chart.js 시각화	결과를 차트로 직관적으로 이해, 프레젠테이션용 시각 자료 확보
시나리오 비교 (메모리)	다양한 설정을 빠르게 비교하여 최적의 구성 도출, LocalStorage 불필요
로그 스케일 속도 슬라이더	0.1x~100x 광범위한 속도를 직관적으로 조절 가능
Re-simulation 방식 되감기	역방향 이벤트 처리 불필요, 엔진 재생성 후 빠른 실행으로 간단히 구현
Cargo 배지 시각화	로봇의 운반 상태를 시각적으로 명확히 전달
반응형 UI (모바일 대응)	현장 태블릿/스마트폰에서 바로 확인 가능

10. 개선 로드맵

구현 완료

기능	완료일
차트/그래프 시각화 (Chart.js 4종 차트)	2026-02-06
시나리오 비교 기능 (최대 5개, 비교표+차트)	2026-02-06
실시간 모드 개선 — Pallet 시각화 + 타임라인 seek + 속도 미세 조절	2026-02-10
모바일 반응형 UI — 태블릿/스마트폰 레이아웃 + 터치 지원 + 인쇄 스타일	2026-02-11

미구현 (향후 검토)

기능	난이도	효과	설명
설정 프리셋 & 저장/불러오기	하	★★★★☆	LocalStorage 기반 설정 관리, JSON 내보내기/가져오기
최적화 제안 시스템	중	★★★☆☆	결과 분석 후 자동 개선안 제시, 병목 지점 감지
각 이벤트 사운드 효과	하	★★☆☆☆	실시간 모드에서 이벤트별 사운드 (옵션)
다중 로봇 지원	상	★★☆☆☆	로봇 2~3대 운용, Zone 분할 / 부하 분산 알고리즘
고급 스케줄링 알고리즘 비교	상	★★☆☆☆	FIFO/SJF/라운드 로빈/동적 우선순위 선택 및 비교
실제 데이터 연동	최상	★★☆☆☆	REST API 연동, 예측 vs 실제 비교 대시보드
단위 테스트	중	★★☆☆☆	Jest/Mocha로 engine.js 테스트, CI/CD